

RAG + LLM + MCP + AI

A Practical Guide for Building AI Knowledge and Agent Systems

Purpose: This document is designed as a realistic knowledge base for a beginner-to-intermediate RAG project. It contains enough structured information to test document loading, chunking, embeddings, vector search, retrieval, and question answering.

Topics covered: RAG, LLMs, tokens, embeddings, vector databases, Qdrant, LangChain, LangGraph, MCP, AI agents, prompts, retrieval strategies, evaluation, hallucination reduction, fine-tuning, security, and a complete project architecture.

Table of Contents

- 1. Artificial Intelligence and Modern AI Systems
- 2. Large Language Models
- 3. Tokens, Context Windows and Inference
- 4. Retrieval-Augmented Generation
- 5. Documents, Chunking and Metadata
- 6. Embeddings and Semantic Search
- 7. Vector Databases and Qdrant
- 8. LangChain for RAG
- 9. Retrieval and Prompt Construction
- 10. MCP and Tool Connectivity
- 11. AI Agents and LangGraph
- 12. RAG + MCP Architecture
- 13. RAG vs Fine-tuning
- 14. Evaluation, Hallucination and Security
- 15. Complete Project Workflow and Interview Questions

1. Artificial Intelligence and Modern AI Systems

Artificial Intelligence refers to software systems that perform tasks that normally require human-like capabilities such as language understanding, prediction, classification, reasoning, perception, and decision support. Modern AI applications often combine several components rather than depending on one model.

A typical production AI system may contain an application layer, an LLM, a retrieval layer, external tools, databases, authentication, monitoring, and evaluation. Understanding how these components interact is important for an AI engineer because an LLM alone does not automatically provide reliable access to private company information.

Generative AI systems create text, code, images, audio, or other content from prompts. LLM applications focus on language and can be connected to external knowledge through retrieval and external actions through tools.

Key engineering points

- AI application: the user-facing software that receives requests and presents results.
- Model layer: an LLM or other machine-learning model that processes information.
- Knowledge layer: documents, databases, APIs, and vector stores containing useful information.
- Tool layer: external capabilities such as search, calculators, databases, or business APIs.
- Orchestration layer: code or frameworks that decide which component should run and in what order.

2. Large Language Models

A Large Language Model is a neural network trained on large collections of text to learn statistical patterns in language. During generation, the model predicts a sequence of tokens conditioned on the input context. Transformer architectures are widely used because attention mechanisms allow the model to relate different parts of a sequence.

An LLM can summarize, classify, explain, generate code, answer questions, and follow instructions. However, its internal knowledge is not a live database. It can also produce confident but unsupported statements, commonly called hallucinations.

For enterprise applications, developers often provide relevant external information in the prompt. This is one reason RAG is useful.

Key engineering points

- Parameters are learned numerical values inside a model.
- Inference is the process of using a trained model to generate an output.
- Temperature can influence randomness in generation.
- Context window determines how much input and conversation information the model can process at once.
- A model can generate an answer from retrieved evidence without changing its parameters.

3. Tokens, Context Windows and Inference

LLMs generally process text as tokens rather than complete words. A token may be a word, part of a word, punctuation mark, or other text unit. Tokenization affects prompt length, cost, and context usage.

The context window is the amount of tokenized information a model can consider in a single request. A RAG system therefore should not blindly send an entire large document to the model. Instead, it retrieves a small number of relevant chunks.

Inference is the runtime stage. A user query is transformed into model input, the model computes probabilities over possible next tokens, and a response is generated.

Key engineering points

- Token count affects model limits and often API cost.
- Chunk size should balance semantic completeness with retrieval precision.
- Large chunks may contain irrelevant material.
- Very small chunks may lose context and make answers incomplete.
- Retrieved context should be selected based on relevance rather than document size.

4. Retrieval-Augmented Generation

Retrieval-Augmented Generation, or RAG, combines information retrieval with language generation. The central idea is simple: first find relevant information from an external knowledge source, then provide that information to an LLM so it can generate an answer grounded in the retrieved evidence.

A basic RAG pipeline begins with documents. Documents are loaded and split into chunks. Each chunk is converted into an embedding vector and stored in a vector database. When a user asks a question, the question is also embedded. Similarity search finds relevant chunks. Those chunks are inserted into a prompt, and the LLM produces the final answer.

RAG is useful when information changes frequently or when the knowledge is private and should not be encoded into model parameters.

Key engineering points

- Ingestion happens before user queries and prepares the knowledge base.
- Retrieval happens at query time.
- Generation uses retrieved context to answer the question.
- Metadata can restrict retrieval to a specific document, department, product, or date.
- RAG does not automatically guarantee truth; retrieval quality and prompt design matter.

5. Documents, Chunking and Metadata

A document loader converts a source such as PDF, HTML, Markdown, CSV, or text into application-readable documents. In LangChain, a document commonly contains page content plus metadata.

Chunking divides long documents into smaller pieces. A chunk should ideally represent a coherent unit of meaning. Overlap can preserve context between neighboring chunks. For example, if a chunk ends in the middle of a concept, an overlapping region can help the next chunk retain context.

Metadata is information attached to a chunk, such as file name, page number, section, author, product, or date. Metadata is valuable for citations, filtering, debugging, and access control.

Key engineering points

- Use chunk sizes appropriate to the model and document structure.
- Prefer semantic boundaries such as headings and paragraphs when possible.
- Keep source metadata so retrieved answers can be traced back to documents.
- Avoid excessive overlap because it increases storage and duplicate retrieval.
- Test chunking using real questions rather than choosing values blindly.

6. Embeddings and Semantic Search

An embedding model converts text into a numerical vector. The vector represents patterns of meaning learned by the embedding model. Texts with related meanings tend to be located closer together according to an appropriate similarity metric.

A query is embedded using the same embedding model used for stored document chunks. The vector database compares the query vector with stored vectors and returns the most relevant results.

Semantic search differs from exact keyword search. A query can retrieve a chunk even when the exact words in the query do not appear in the document. Embedding quality is therefore a critical part of RAG quality.

Key engineering points

- Cosine similarity is commonly used to compare vector direction.
- Euclidean distance compares geometric distance.
- Dot product is another similarity measure used by some systems.
- The embedding dimension must match the vector collection configuration.
- Changing the embedding model generally requires re-embedding the stored data.

7. Vector Databases and Qdrant

A vector database stores embeddings and supports similarity search. Qdrant is a vector database that can store vectors together with payload data and metadata. In a RAG system, each stored point can represent a document chunk.

A collection defines a space for vectors with a configured vector size and distance metric. Payloads can store information such as the original text, source file, page number, or category.

A typical retrieval request asks Qdrant for the top-k points most similar to a query vector. The application then extracts the payload text and supplies it to the LLM.

Key engineering points

- Collection: logical storage area for vectors.
- Vector: numerical representation of a chunk.
- Payload: metadata and associated information.
- Top-k: number of highest-ranked results returned.
- Filtering: restricts search using payload conditions.
- Qdrant Cloud provides a managed hosted option for applications.

8. LangChain for RAG

LangChain provides reusable components for building LLM applications. For a basic RAG application, useful components include document loaders, text splitters, embedding integrations, vector stores, retrievers, prompt templates, and chat model integrations.

LangChain does not replace the underlying vector database or LLM. It provides an orchestration layer that makes it easier to connect these components. Qdrant can be integrated through the LangChain Qdrant integration package.

A simple chain can load documents, split them, create embeddings, store them in Qdrant, retrieve relevant chunks, construct a prompt, and call an LLM.

Key engineering points

- Document loaders handle source-specific parsing.
- Text splitters prepare documents for retrieval.
- Embeddings convert text into vectors.
- Vector stores provide storage and similarity search.
- Retrievers expose a standard way to fetch relevant documents.
- Prompt templates define how retrieved context is presented to the model.

9. Retrieval and Prompt Construction

Retrieval quality determines which evidence reaches the LLM. A basic similarity retriever can be improved using metadata filters, score thresholds, reranking, query transformation, or hybrid search.

The prompt should clearly separate instructions from retrieved evidence. A good RAG prompt tells the model to answer using the supplied context and to say when the context does not contain enough information.

Top-k should not be chosen solely by intuition. Too few results can miss important evidence, while too many can introduce noise and consume context space.

Key engineering points

- Retrieve multiple chunks when an answer requires information from different sections.
- Use source metadata to produce useful citations or references.
- Consider reranking when initial vector search returns several loosely relevant chunks.
- Use score thresholds when low-similarity results should be rejected.
- Design prompts that discourage unsupported answers.

10. MCP and Tool Connectivity

Model Context Protocol, commonly called MCP, is a protocol for connecting AI applications with external tools and data sources in a standardized way. An MCP server can expose tools, resources, or prompts to an MCP-compatible client.

MCP is not the same thing as RAG. RAG is a retrieval pattern for supplying relevant information to a model. MCP is a communication protocol that can expose external capabilities to an AI application. They can be combined.

For example, an MCP server could expose a company database search tool. The AI application can call that tool when appropriate, receive structured information, and then use the result in its reasoning or final response.

Key engineering points

- MCP client: application that connects to MCP servers.
- MCP server: exposes tools, resources, or prompts.
- Tool: an executable capability such as a database query or API action.
- Resource: externally available information exposed through the protocol.
- MCP can help standardize how AI applications discover and invoke external capabilities.

11. AI Agents and LangGraph

An AI agent is an application in which a model can decide which actions or tools are needed to complete a task. Unlike a simple linear chain, an agent can involve branching, iteration, tool calls, validation, and state.

LangGraph is designed for building stateful agent workflows as graphs. Nodes perform actions and edges determine transitions. A graph can include an LLM node, retrieval node, tool node, validation node, and response node.

For a beginner RAG project, a simple chain is easier to understand. Once retrieval works reliably, the system can be extended into a graph or agent.

Key engineering points

- A chain generally follows a predetermined sequence.
- An agent can choose among tools based on the request.
- A graph can model conditional paths and repeated steps.
- State stores information needed across steps.
- Validation nodes can check whether retrieved evidence supports an answer.

12. RAG + MCP Architecture

RAG and MCP can complement each other. A RAG system may use a vector database as its knowledge store, while MCP can expose additional live tools such as database search, ticket lookup, inventory queries, or internal APIs.

A possible architecture is: user query → AI application → intent decision → retrieve documents from Qdrant and/or call an MCP tool → combine evidence → LLM → final answer. The application should control permissions and validate tool inputs.

This architecture is especially useful when some information is static in documents while other information is dynamic in operational systems.

Key engineering points

- Static policy documents can be indexed in Qdrant.
- Live order status can be retrieved from an MCP-connected business system.
- The LLM can combine retrieved evidence from both sources.
- Authentication and authorization should happen outside the model's free-form reasoning.
- Tool results should be treated as data, not blindly trusted instructions.

13. RAG vs Fine-tuning

RAG and fine-tuning solve different problems. RAG changes the information supplied to a model at inference time. Fine-tuning changes model parameters through additional training.

Use RAG when the goal is to provide external knowledge, private documents, changing information, or traceable source material. Fine-tuning can be useful when the desired behavior, style, output format, or task-specific pattern needs to be learned.

They can also be combined. A fine-tuned model can use RAG for external knowledge, but fine-tuning should not normally be the first solution for simply adding a collection of changing documents.

Key engineering points

- RAG: external knowledge at runtime.
- Fine-tuning: modifies model behavior or learned parameters.
- RAG knowledge can be updated by changing the index.
- Fine-tuning requires a training dataset and training process.
- Choose based on the problem rather than treating one technique as universally better.

14. Evaluation, Hallucination and Security

A production RAG system should be evaluated rather than judged by a few successful examples.

Evaluation can measure retrieval relevance, answer correctness, faithfulness to retrieved context, latency, and failure behavior.

Hallucination can happen when the retriever returns irrelevant evidence, when the context is incomplete, or when the model generates unsupported content. A useful strategy is to instruct the model to distinguish supported answers from unknown information and to evaluate retrieval separately from generation.

Security is equally important. Documents may contain sensitive information, and tool calls may perform real actions. API keys should be stored in environment variables or secret managers and never committed to source control.

Key engineering points

- Create a test set of representative questions.
- Measure whether the correct source chunk appears in retrieved results.
- Check whether the final answer is supported by retrieved evidence.
- Use least-privilege permissions for database and tool credentials.
- Treat retrieved documents and tool output as untrusted data.
- Log failures without exposing secrets or sensitive document content.

15. Complete Project Workflow and Interview Questions

A practical beginner RAG project can be built incrementally. First, create a virtual environment and install the required packages. Next, connect to Qdrant Cloud and verify access. Then load a PDF using LangChain, split the content into chunks, create embeddings, and store the vectors in Qdrant. Finally, create a retriever, build a prompt, call an LLM, and return an answer based on retrieved context.

After the basic pipeline works, add metadata, citations, evaluation, conversation history, reranking, and optionally MCP tools. This progression makes debugging easier because each component can be tested independently.

Key engineering points

- Phase 1: Environment setup and dependency installation.
- Phase 2: Qdrant Cloud connection test.
- Phase 3: PDF loading.
- Phase 4: Chunking and metadata creation.
- Phase 5: Embedding generation.
- Phase 6: Qdrant collection and vector insertion.
- Phase 7: Retriever and similarity search.
- Phase 8: Prompt construction and LLM generation.
- Phase 9: Evaluation and debugging.
- Phase 10: Optional MCP and agent integration.

Interview Questions for This Project

What is RAG?

A pattern that retrieves relevant external information and supplies it to a generative model before answering.

Why use a vector database?

To store embeddings and efficiently search for semantically similar content.

Why is chunking required?

Large documents are divided into retrievable units so relevant information can be selected efficiently.

What is an embedding?

A numerical vector representation of text used for semantic similarity.

Why Qdrant?

It provides vector storage, similarity search, payload metadata, and filtering for retrieval applications.

What is LangChain?

A framework providing components and orchestration for LLM applications.

What is MCP?

A protocol for connecting compatible AI applications with external tools and resources.

RAG vs fine-tuning?

RAG supplies external knowledge at runtime; fine-tuning changes model parameters through training.

What causes hallucination in RAG?

Poor retrieval, incomplete context, ambiguous prompts, or unsupported generation.

How would you improve retrieval?

Improve chunking, embeddings, metadata filters, top-k selection, hybrid search, or reranking.

Glossary

Term	Meaning
AI	Artificial Intelligence
LLM	Large Language Model
RAG	Retrieval-Augmented Generation
Embedding	Vector representation of information
Vector DB	Database optimized for vector similarity search
Qdrant	Vector database used for semantic retrieval
Chunk	Smaller section of a source document
Retriever	Component that fetches relevant documents
Prompt	Instructions and context supplied to a model
MCP	Model Context Protocol
Agent	AI application capable of selecting actions/tools
LangGraph	Framework for stateful graph-based AI workflows
Metadata	Information describing a stored document/chunk
Top-k	Number of highest-ranked retrieval results